

Agentic Healthcare Assistant for Medical Task Automation

Applied Generative AI Specialization — Capstone Writeup

Author	Jeeth Sellamuthu
Project	Agentic Healthcare Assistant (HCAsst)
Stack	LangGraph · FAISS · SentenceTransformers · Claude / MockLLM · SQLite · Streamlit
Modes	Offline-first (deterministic) · Online (Anthropic Claude)

1. Problem and Objectives

Clinical staff spend significant time on repetitive coordination tasks: scheduling, updating records, summarizing histories, and looking up disease information. This project builds an **agentic** assistant that automates these four task families while honoring the constraints that make a medical tool trustworthy — no fabricated clinical claims, human approval on every write, protection of patient data, and reliance only on trusted sources.

Four target services

- **Book appointments** — discover slots and propose scheduling from patient intent and provider availability.
- **Manage records** — add or update structured and unstructured patient history.
- **Retrieve histories** — summarize diagnoses, treatments, and alerts using an LLM over retrieved evidence.
- **Search disease info** — fetch up-to-date information from trusted external sources (MedlinePlus, PubMed).

Quality bars (non-negotiable)

- **Zero-hallucination:** every clinical statement carries a citation; unsupported claims are regenerated or caveated.
- **Human-in-the-loop (HITL):** writes are proposed, then confirmed by a human — never auto-executed.
- **PHI-safe:** identifiers are masked before any log or trace is persisted (HIPAA-aligned).
- **Reproducible:** the system runs fully offline with a deterministic mock model — no key or network required.

2. Architecture

The assistant is a **LangGraph state machine**. A planner decomposes the query into an ordered set of tool steps; a router executes them, looping until the plan is exhausted; a grounding guardrail verifies the output; and the response is assembled, returned, and committed to long-term memory. Every node writes a PHI-masked trace, which is what makes per-module observability possible.

```
→ identify → route ■ { summarize | disease | book } → guardrail → respond → memory
```

Four design decisions (the strategy)

- **State machine, not a single prompt.** The query is multi-intent and stateful; discrete nodes are inspectable, loggable, and guardable — the architecture and the observability requirement are one decision.
- **RAG + citation guardrail, not blind trust.** Retrieval supplies real evidence; the guardrail regenerates an answer up to three times if claims are uncited, then appends a caveat rather than emit an unverifiable statement.
- **Offline-first with a MockLLM.** A deterministic mock backend makes the project reproducible for grading and demoable without a key; it also forces a clean, swappable LLM abstraction.
- **HITL on writes.** The gap between *propose* and *confirm* is the guardrail; the agent proposes, a human commits.

Guiding principle

Program to the interface, not the implementation. The LLM provider, the embedder, and the disease-search backend each sit behind a stable contract with two implementations (real and offline/fallback). This is why online/offline and mock/real swap without touching any caller — and it is what allowed the instructor dataset to be ingested as a thin adapter rather than a rewrite.

3. Implementation

Component	Technology	Role
Orchestration	LangGraph	Agent state machine; conditional routing loop
Planning	LLM + schema validation	Decompose query into a validated tool plan
Retrieval (RAG)	FAISS + SentenceTransformers	Semantic retrieval of evidence (all-MiniLM-L6-v2, 384-d)
Memory	SQLite + FAISS	Long-term context; semantic recall across sessions
LLM provider	Claude / MockLLM	Swappable behind one Protocol; offline = deterministic
Guardrail	Regex + LLM-judge	Citation check + grounding; regenerate-or-caveat (loop of 3)
Tools	Scheduling, Records, Disease	HITL booking; EHR add/update; MedlinePlus + PubMed
Persistence	SQLite	Appointments, audit, traces, memory, eval runs
Observability	PHI masking	Identifiers masked before any trace is stored
Evaluation	QAEval + metrics	Tool-F1, booking, citation coverage, grounding
UI	Streamlit + Plotly	Six views: Assistant, Patient, Doctor, Eval, Traces, Docs

Sample scenario (end to end)

“My 70-year-old father has chronic kidney disease. I want to book a nephrologist for him. Also, can you summarize the latest treatment methods?”

The planner decomposes this into `get_patient_history` → `book_appointment` → `summarize_history` → `search_disease_info`. The agent loads the patient, proposes a nephrology booking (held for approval), produces a cited history summary, and synthesizes cited treatment information — passing the grounding check before assembling the response.

4. Requirement Coverage

Spec requirement	Where it is met
Booking — slot discovery + scheduling	tools/scheduling.py; HITL propose→confirm; Doctor view
Records — add/update structured & unstructured	tools/records.py; Patient view form; audited
Retrieve & summarize histories (LLM)	agent/summarize.py; cited RAG summary
Disease search (Medline / trusted sources)	tools/disease_search.py; MedlinePlus + PubMed + offline KB
Doctor schedule API	Synthetic scheduler (slots + roster); FHIR is stretch
EHR / Patient DB	SQLite + seed + instructor xlsx/PDF ingestion
FAISS vector DB for summaries	rag/vectorstore.py (IndexFlatIP) + SentenceTransformers
Long-term memory module	memory/store.py; durable + semantic recall
Structured prompts + task chaining	llm/prompts.py; plan→summarize→search→ground chain
Patient context via memory lookups	memory recall injected into summarization
Sample CKD scenario, end to end	agent/graph.py; golden eval case
Evaluation (QAEval + metrics)	eval/ — tool-F1, booking, citation coverage, grounding
Dashboard + traces + tool logs	app/streamlit_app.py — six views
PHI-safe logging	obs/logging.py masking; traces log references, not content

Human-in-the-loop	Booking and record writes both propose→confirm
-------------------	--

5. Instructor Dataset Integration

The course dataset ships patient data as a roster spreadsheet (`records.xlsx`) and SOAP-format PDF reports (Anjali Mehra, David Thompson, Ramesh Kulkarni). A ~150-line adapter (`tools/ingest_external.py`) parses both into the same typed `PatientHistory` / `EvidenceItem` models the agent already uses. Because every downstream component depends on those models rather than on a data source, retrieval, summarization, grounding, and booking all worked on real data unchanged.

This integration also exercises the PHI-safety layer against real identifiers: the reports contain dates of birth (MM/DD/YYYY), Indian-format phone numbers (+91-...), addresses, and emails. Masking preserves the birth year (so age remains computable) while redacting month, day, phone, email, and SSN. A known limitation is that regex masking does not catch free-text names or addresses embedded in prose; production would add named-entity recognition for those.

6. Evaluation

An LLMOps harness scores the agent over four golden cases (CKD+nephrology, AFib+cardiology, summary-only, disease-only) and persists results to the `eval_runs` table for the dashboard. Metrics: tool-selection F1, booking success, citation coverage, QA correctness (LLM-judge online / token-overlap offline), and grounding rate.

Metric	Offline (mock)	Note
<code>planning.tool_f1</code>	0.95	Plan matches expected tools
<code>booking.success</code>	1.00	Correct specialty proposed, HITL respected
<code>grounding.grounded</code>	1.00	All answers cite evidence
<code>citation_coverage</code>	0.50	Mock text has few cited lines (artifact)
<code>qa_correctness</code>	0.00-0.25	Mock returns templated text; valid only vs. real LLM

Tool selection, booking, and grounding are valid offline; answer-quality metrics (QA correctness, citation coverage) are representative only against the real Claude backend, by design — the offline mock trades answer richness for determinism.

7. Limitations and Future Work

- **Live FHIR/HL7** read-write is stubbed by seed + instructor data with identical return types; wiring a real EHR is the next step.
- **Adaptive memory** (Mem0/Zep-style consolidation, supersede, salience) is deferred; the current store is durable but simple.
- **PHI masking** covers structured identifiers; named-entity recognition for free-text names/addresses is future work.
- **CCD parsing** for the most complex report is intentionally minimal (diagnosis + meds); a robust CCD parser is its own project.
- **Booking** uses a synthetic provider roster and slots; integration with a real calendar API is straightforward via the same interface.

Summary

The assistant meets every functional requirement of the brief and all four quality bars, runs reproducibly offline, upgrades to real Claude with a key, and operates on the official instructor dataset. The architecture's central idea — programming to stable interfaces — is what made it extensible, testable, and defensible.